Linear regression and regularisation: least squares, ridge and lasso

Prof. Dr. Emre Onur Kahya · Department of Physics Engineering, Istanbul Technical University

Linear regression is the simplest model that is learned by empirical risk minimisation and the building block of every neural network: a single linear neuron. This note derives the least-squares solution and its geometry, shows when it becomes unstable, and introduces the two classical regularisers — ridge (L_2) and lasso (L_1) — with their closed-form or iterative solutions and their Bayesian reading. Linear probes of language-model activations and the sparse discovery of physical laws illustrate how much this small model still does.

1 The model

A linear (strictly, affine) regressor predicts

$$f_\theta(\mathbf{x}) = \mathbf{w}^\mathsf{T}\mathbf{x} + b = w_1 x_1 + \cdots + w_d x_d + b, \qquad \theta = (\mathbf{w}, b),\ \mathbf{w} \in \mathbb{R}^d,\ b \in \mathbb{R}.$$

Each input has a weighted influence on the output: w_j is the change in the prediction when x_j grows by one unit, all else fixed. In neural-network language this is a single output neuron connected to d input neurons by *synaptic weights* w_j, with a *bias* b and no non-linearity. It is a good habit to absorb the bias by prepending a constant feature: with $\tilde{\mathbf{x}} = (1, x_1, \dots, x_d)^\mathsf{T}$ and $\tilde{\mathbf{w}} = (b, w_1, \dots, w_d)^\mathsf{T}$ we have $f(\mathbf{x}) = \tilde{\mathbf{w}}^\mathsf{T}\tilde{\mathbf{x}}$. Stacking the $\tilde{\mathbf{x}}_i^\mathsf{T}$ as rows gives the $n \times (d+1)$ matrix $\tilde{X}$, the design matrix of Note 1 with a column of ones in front, and the targets form $\mathbf{y} \in \mathbb{R}^n$. We drop the tildes on $\mathbf{w}$ when no confusion arises.

The model is linear in its *parameters*, not necessarily in the original measurements: $\mathbf{x}$ may contain logarithms, products or powers of raw quantities. Polynomial regression (Note 4) is linear regression on the features $(x, x^2, \dots, x^p)$, a point Note 9 develops.

2 Least squares

With the squared loss the empirical risk (up to the factor $1/n$) is

$$J(\mathbf{w}) = \sum_{i=1}^{n} (\mathbf{w}^\mathsf{T}\tilde{\mathbf{x}}_i - y_i)^2 = \left\| \tilde{X}\mathbf{w} - \mathbf{y} \right\|^2 .$$

It is a convex quadratic in $\mathbf{w}$ with gradient $\nabla J = 2\tilde{X}^\mathsf{T}(\tilde{X}\mathbf{w} - \mathbf{y})$. Setting it to zero gives the *normal equations*

$$\tilde{X}^\mathsf{T}\tilde{X}\,\mathbf{w} = \tilde{X}^\mathsf{T}\mathbf{y}, \qquad \hat{\mathbf{w}} = (\tilde{X}^\mathsf{T}\tilde{X})^{-1}\tilde{X}^\mathsf{T}\mathbf{y} \quad \text{if } \tilde{X}^\mathsf{T}\tilde{X} \text{ is invertible.} \tag{1}$$

Note 2, Section 3, showed that minimising squared error is maximum likelihood under Gaussian noise, so $\hat{\mathbf{w}}$ is also the maximum-likelihood estimate of that model.

Geometry. The fitted values $\hat{\mathbf{y}} = \tilde{X}\hat{\mathbf{w}}$ lie in the column space of $\tilde{X}$, the set of all predictions the model can make. The normal equations say $\tilde{X}^{\mathsf{T}}(\mathbf{y} - \hat{\mathbf{y}}) = \mathbf{0}$: the residual is orthogonal to every column, so $\hat{\mathbf{y}}$ is the orthogonal projection of $\mathbf{y}$ onto that space, the closest point to the observations. One consequence: with a constant column, the residuals sum to zero.

Computing it. Never form the inverse in (1). Solve the least-squares problem with a QR or singular value decomposition (`np.linalg.lstsq`), at a cost of $O(nd^2)$; squaring $\tilde{X}$ into $\tilde{X}^{\mathsf{T}}\tilde{X}$ squares its condition number and loses half the significant digits. For very large n or d, gradient descent (Note 7) needs only $O(nd)$ work per pass, and its learning rate is limited by the largest eigenvalue of $2\tilde{X}^{\mathsf{T}}\tilde{X}/n$; mini-batch gradients make it work on data that do not fit in memory.

3 When least squares is unstable

The solution is unique only if the columns of $\tilde{X}$ are linearly independent, and it is reliable only if they are far from dependent.

- *Collinear features.* If two inputs measure almost the same thing, the data cannot tell how to split the weight between them: $w_1 + w_2$ is well determined, $w_1 - w_2$ is not. Figure 1 shows 300 repetitions of an experiment with $x_2 = x_1 +$ small noise: least-squares estimates of w_1 scatter with a standard deviation of 2.05 around the true value 1, along a line of nearly constant $w_1 + w_2$. Predictions inside the data remain fine; interpretations and predictions away from the data do not.
- *More features than examples.* If $d + 1 > n$, $\tilde{X}^{\mathsf{T}}\tilde{X}$ is singular and infinitely many $\mathbf{w}$ fit the training data exactly. `lstsq` then returns the one of minimum norm — the implicit preference behind the double-descent curve of Note 4, Section 8.

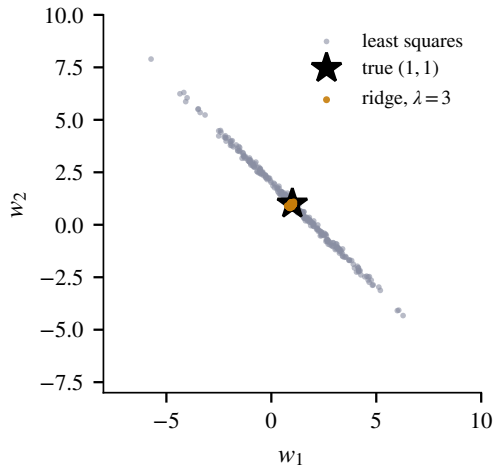


Figure 1. Estimates of (w_1, w_2) from 300 datasets of 30 points with nearly collinear inputs and true weights $(1, 1)$. Least squares (grey) spreads along the poorly determined direction $w_1 - w_2$; ridge regression with $\lambda = 3$ (amber) stays close to the truth: standard deviation of w_1 falls from 2.05 to 0.05, at the price of a small bias (mean 0.95).

4 Ridge regression

Add the weight-decay penalty of Note 4, Section 7:

$$J_\lambda(\mathbf{w}) = \left\|\tilde{X}\mathbf{w} - \mathbf{y}\right\|^2 + \lambda \sum_{j=1}^{d} w_j^2, \tag{2}$$

leaving the bias unpenalised (shifting all targets by a constant should shift b, not shrink it). Setting the gradient to zero gives

$$\hat{\mathbf{w}}_\lambda = (\tilde{X}^\mathsf{T}\tilde{X} + \lambda\check{I})^{-1}\tilde{X}^\mathsf{T}\mathbf{y},$$

where $\check{I}$ is the $(d+1)\times(d+1)$ identity with its first diagonal entry, the one for the bias, set to zero. For $\lambda > 0$ the matrix is positive definite, so the solution is unique even when $d > n$ [6].

What ridge does. Centre the inputs and targets (which removes the bias) and write the singular value decomposition $X = USV^\mathsf{T}$ with singular values $s_1 \geq s_2 \geq \dots$. Then

$$\hat{\mathbf{y}}_\lambda = \sum_j \mathbf{u}_j \frac{s_j^2}{s_j^2 + \lambda}\,\mathbf{u}_j^\mathsf{T}\mathbf{y}.$$

Least squares ($\lambda = 0$) keeps every direction of the data; ridge shrinks each one by $s_j^2/(s_j^2+\lambda)$: directions in which the inputs vary a lot are kept, directions of little variation — where the estimate was noise, as for $w_1 - w_2$ in Figure 1 — are suppressed. The sum $\sum_j s_j^2/(s_j^2+\lambda)$ is the *effective number of parameters*, which decreases continuously from d to 0 as λ grows: the effective capacity of Note 4, Section 5, made exact. Because the penalty treats all w_j alike, features must be put on comparable scales first (Note 5, Section 5); otherwise λ penalises a coefficient merely for its units.

5 The lasso and sparse models

Replacing the squared penalty with absolute values gives the *lasso* [7],

$$J_\lambda(\mathbf{w}) = \left\|\tilde{X}\mathbf{w} - \mathbf{y}\right\|^2 + \lambda\sum_{j=1}^{d}|w_j|.$$

The L_1 penalty has a kink at zero, and for large enough λ many coefficients become exactly zero: the lasso selects features. There is no closed form, but coordinate descent is simple and fast: updating one w_j with the others fixed is a one-dimensional problem whose solution is a *soft threshold* — the least-squares value moved towards zero by $\lambda/2$ (in units of $\left\|X_{:,j}\right\|^2$, the squared norm of column j), and set to zero if it would cross it. The *elastic net* adds both penalties, keeping the sparsity of the lasso while behaving better with groups of correlated features [8].

Figure 2 follows the coefficients of eight correlated features, of which three matter, as λ grows. Ridge shrinks all of them smoothly and none reaches zero. The lasso removes the irrelevant ones first: eight non-zero coefficients at $\lambda = 1$, five at $\lambda = 10$, four at 30 and only the three true ones at $\lambda = 100$. Which λ to use is decided on validation data (Note 4, Section 7).

6 The Bayesian reading

Model the targets as $y_i = \mathbf{w}^\mathsf{T}\tilde{\mathbf{x}}_i + \varepsilon_i$ with Gaussian noise of variance σ^2, and put a prior on the weights. The *maximum a posteriori* (MAP) estimate maximises $p(\mathbf{w} \mid \text{data}) \propto p(\text{data} \mid \mathbf{w})\,p(\mathbf{w})$, i.e. minimises

$$\frac{1}{2\sigma^2}\left\|\tilde{X}\mathbf{w} - \mathbf{y}\right\|^2 - \log p(\mathbf{w}).$$

A Gaussian prior $w_j \sim \mathcal{N}(0,\tau^2)$ gives $-\log p(\mathbf{w}) = \|\mathbf{w}\|^2/(2\tau^2)+$ const, i.e. ridge regression with $\lambda = \sigma^2/\tau^2$: strong regularisation means a narrow prior relative to the noise. A Laplace prior $p(w_j) \propto$

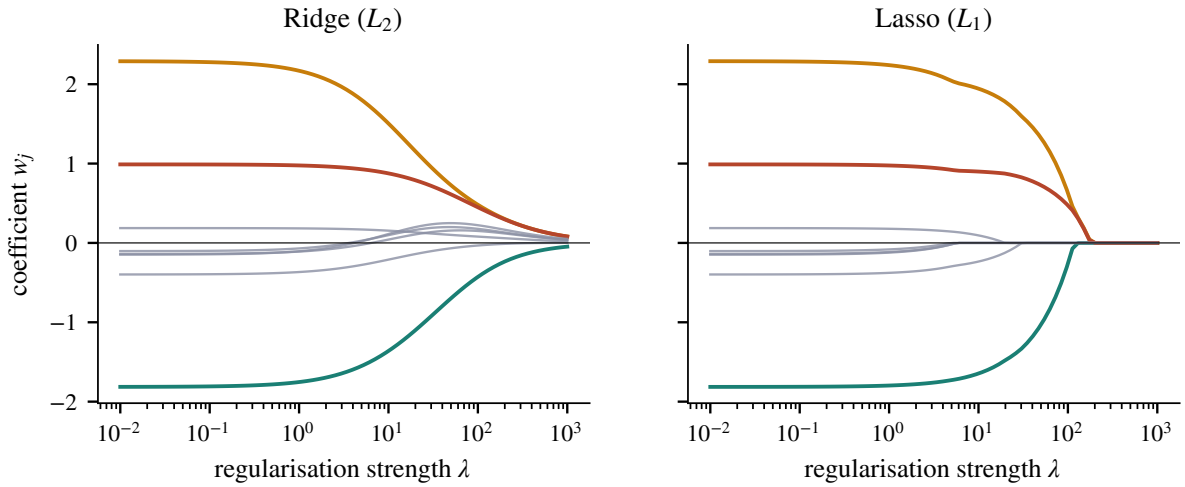


Figure 2. Regularisation paths for $n = 60$ examples with eight standardised, correlated features; coloured lines are the three features with non-zero true weights $(2, -1.5, 1)$, grey lines the five irrelevant ones.

$e^{-|w_j|/\beta}$ gives the lasso with $\lambda = 2\sigma^2/\beta$; its sharp peak at zero is what produces exact zeros. A regulariser is a prior belief about the parameters, made explicit.

Modern connection — linear probes read out what a network knows

Ridge regression is the standard instrument for asking what information a trained network's activations contain: record the activation vector $\mathbf{x}$ of some layer for many inputs, regress a quantity of interest on it, and evaluate on held-out inputs. Applied to the activations of large language models reading the names of places and historical events, such *linear probes* predict latitude, longitude and dates well, suggesting that these quantities are represented along roughly linear directions [3]. A good probe fit shows that information is *present*, not that the network *uses* it — the distinction between probes and causal tests that week 6 of this course takes up.

In physics — discovering equations by sparse regression

To find the law $\dot{\mathbf{z}} = F(\mathbf{z})$ behind a measured trajectory, build a large library of candidate terms — $1, z_1, z_2, z_1^2, z_1 z_2, \sin z_1, \ldots$ — evaluated along the data, and regress the measured derivatives on it with a sparsity-promoting method. Most coefficients vanish, and the few that remain form a readable equation. With this approach ("sparse identification of nonlinear dynamics") the Lorenz equations were recovered from simulated trajectories [2]. Linear regression on nonlinear features, with a sparsity prior playing the role of Occam's razor, becomes a tool for model discovery.

Summary Linear regression predicts $\mathbf{w}^{\mathsf{T}}\mathbf{x} + b$; least squares solves the normal equations, i.e. projects the targets onto the column space of the design matrix, and is the maximum-likelihood fit under Gaussian noise. It becomes unstable with collinear features and undetermined with $d \geq n$. Ridge adds $\lambda \|\mathbf{w}\|^2$, shrinks poorly determined directions and always has a unique closed-form solution; the lasso adds $\lambda \|\mathbf{w}\|_1$ and sets coefficients exactly to zero. In Bayesian terms they are MAP estimates under Gaussian and Laplace priors.

Exercises

E1. Derive the normal equations from $\nabla J = \mathbf{0}$ and show that with a column of ones the residuals sum to zero. Show that in one dimension the slope is $\widehat{\mathrm{Cov}}(x, y)/\widehat{\mathrm{Var}}(x)$.

E2. Derive the ridge solution, including the matrix $\check{I}$. Show that $\tilde{X}^{\mathsf{T}}\tilde{X} + \lambda\check{I}$ is positive definite for every $\lambda > 0$, even when $d > n$. (Consider $\mathbf{v}^{\mathsf{T}}(\tilde{X}^{\mathsf{T}}\tilde{X} + \lambda\check{I})\mathbf{v}$ separately for $\mathbf{v}$ with and without non-zero weight components.)

E3. Using the SVD, show that the ridge fitted values are $\sum_j \mathbf{u}_j s_j^2/(s_j^2 + \lambda)\,\mathbf{u}_j^{\mathsf{T}}\mathbf{y}$ for centred data, and compute the effective number of parameters for singular values $(10, 3, 1, 0.1)$ and $\lambda = 1$.

E4. Show that the minimiser of $(w - a)^2 + \lambda|w|$ is $\mathrm{sign}(a)\max(|a| - \lambda/2, 0)$, and explain why the L_2 penalty never produces exact zeros.

E5. Reproduce Figure 1. Then fit least squares with gradient descent from $\mathbf{w} = \mathbf{0}$ and stop after k steps: compare the spread of the estimates with that of ridge. Which k behaves like $\lambda = 3$?

E6. The period T of a pendulum is measured for several lengths ℓ. Which linear regression, on which transformed variables, estimates the exponent in $T \propto \ell^{\alpha}$? What noise assumption does least squares then make?

Further reading. Hastie, Tibshirani and Friedman [4], chapter 3; Bishop [1], chapter 3; for sparsity, Hastie, Tibshirani and Wainwright [5].

References

[1] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer (2006).

[2] S. L. Brunton, J. L. Proctor and J. N. Kutz, Discovering governing equations from data by sparse identification of nonlinear dynamical systems, *Proc. Natl. Acad. Sci. USA* 113, 3932–3937 (2016).

[3] W. Gurnee and M. Tegmark, Language models represent space and time, *ICLR* (2024).

[4] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer (2009).

[5] T. Hastie, R. Tibshirani and M. Wainwright, *Statistical Learning with Sparsity: The Lasso and Generalizations*, CRC Press (2015).

[6] A. E. Hoerl and R. W. Kennard, Ridge regression: biased estimation for nonorthogonal problems, *Technometrics* 12, 55–67 (1970).

[7] R. Tibshirani, Regression shrinkage and selection via the lasso, *J. R. Stat. Soc. B* 58, 267–288 (1996).

[8] H. Zou and T. Hastie, Regularization and variable selection via the elastic net, *J. R. Stat. Soc. B* 67, 301–320 (2005).

The selection and order of topics in these notes follow the course IFT 6390 *Fondements de l'apprentissage machine* at Université de Montréal (taught by Ioannis Mitliagkas, slides by Pascal Vincent). The text, examples, figures and exercises of this note are original. Figures are computed from synthetic data by `figures.py`, published with the sources of these notes.